

Key Insights

1. Artificial Neurons can be interconnected to form networks leading to more complex machine learning model "Representations"
2. Nonlinear activation functions combined with multiple layers of neurons enable complex data transformations.
3. By optimizing the weights of neurons to reduce the loss function, the network can learn complex representations from the data
4. These transformations alter how the input data is represented within the network and thus redefine the distance or similarity between data points.

Universal approximation theorem: https://en.wikipedia.org/wiki/Universal_approximation_theorem

Delalleau, Olivier, and Yoshua Bengio. "Shallow vs. Deep Sum-Product Networks." In Advances in Neural Information Processing Systems, Vol. 24. Curran Associates, Inc., 2011.

https://papers.nips.cc/paper_files/paper/2011/hash/8e6b42f1644ecb1327dc03ab345e618b-Abstract.html.

Tinker With a **Neural Network** Right Here in Your Browser.

Don't Worry, You Can't Break It. We Promise.



Epoch
000,246

Learning rate

0.03

Activation

Tanh

Regularization

None

Regularization rate

0

Problem type

Classification

DATA

Which dataset do you want to use?



Ratio of training to test data: 50%



Noise: 0



Batch size: 10



REGENERATE

FEATURES

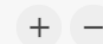
Which properties do you want to feed in?



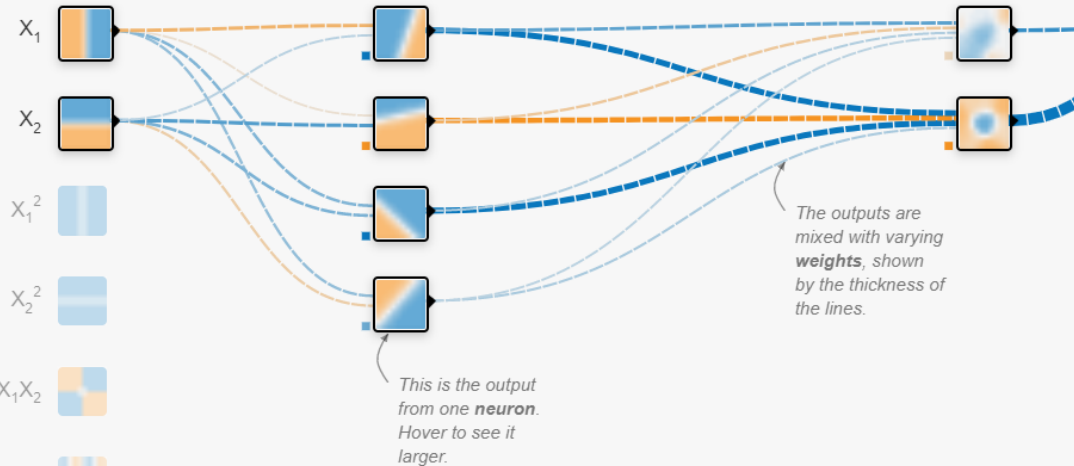
+ - 2 HIDDEN LAYERS



4 neurons



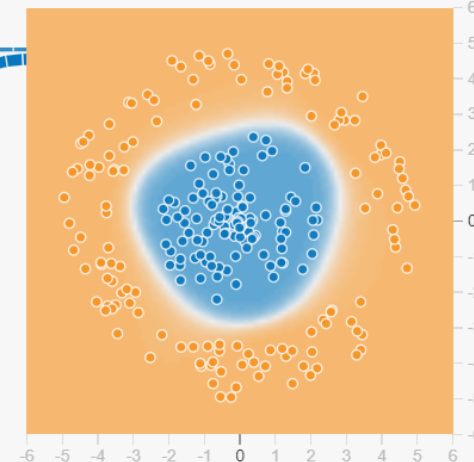
2 neurons



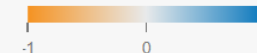
OUTPUT

Test loss 0.003

Training loss 0.001



Colors shows data, neuron and weight values.



☐ Show test data

☐ Discretize output

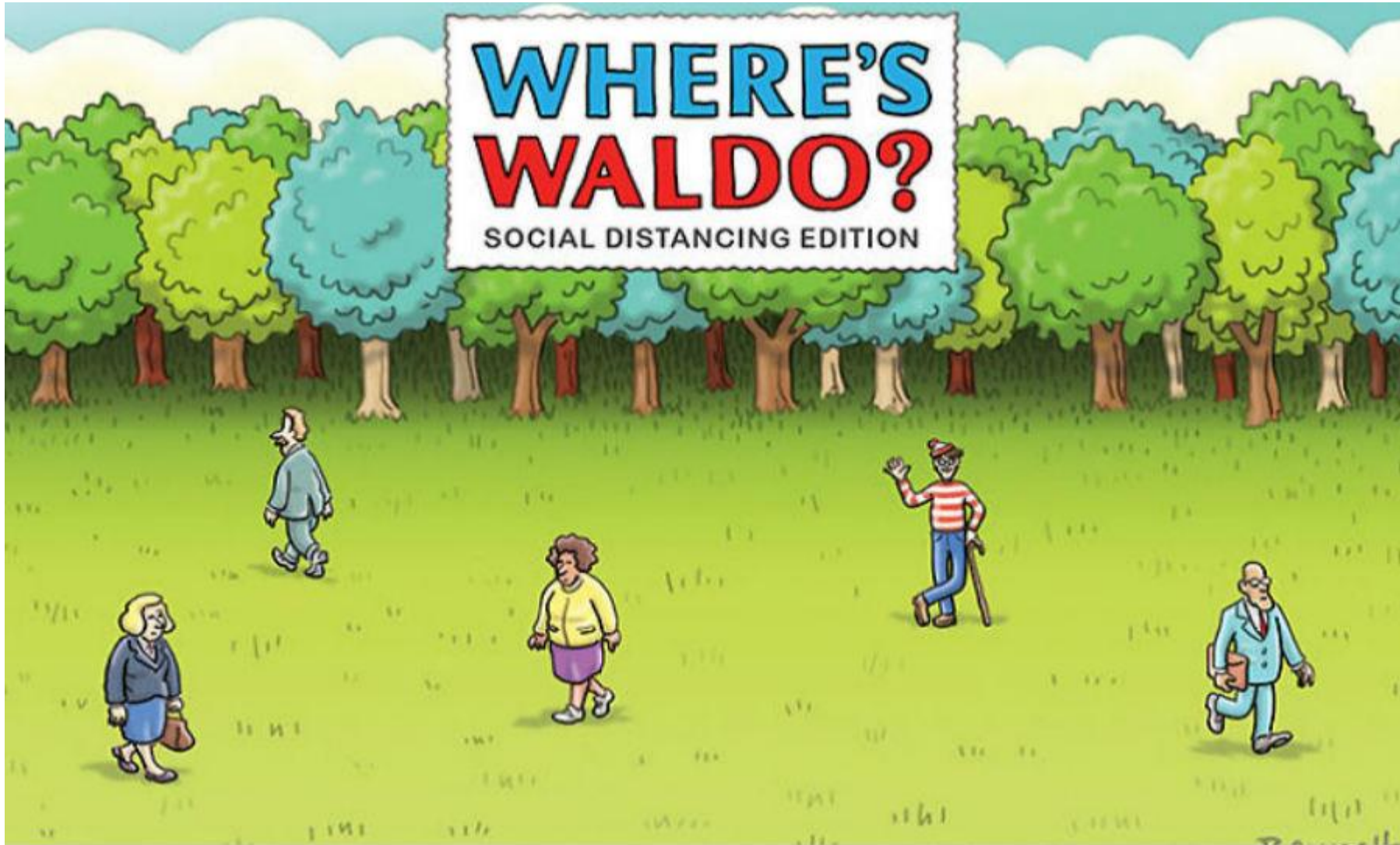
<https://playground.tensorflow.org>

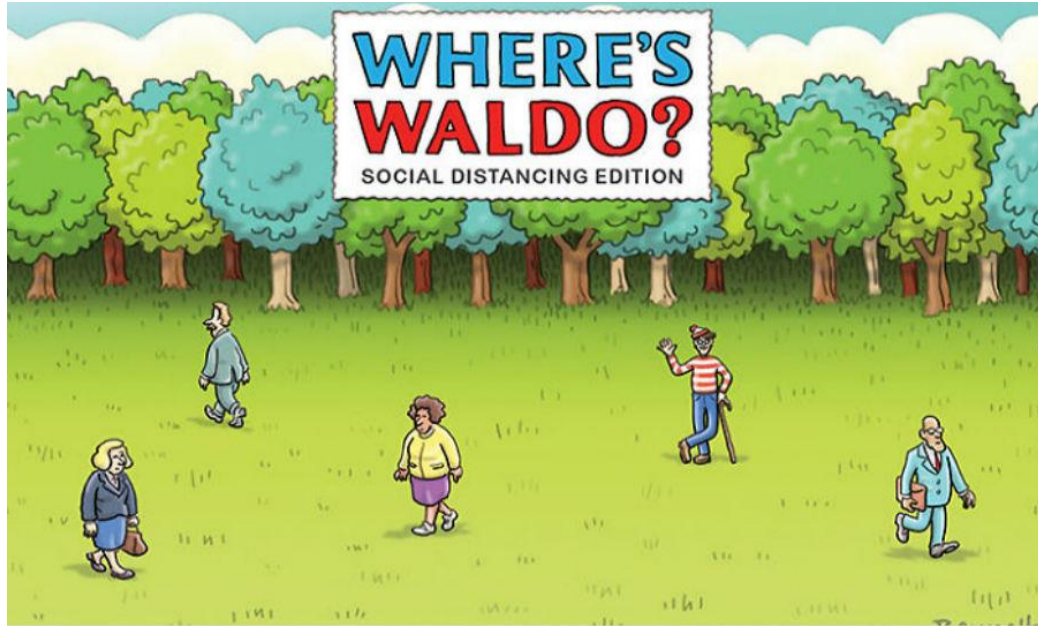
Lab Exercise: Solve XOR and Digit Classification

- Understand the MLP and adapt it to solve XOR Classification
- https://github.com/foxtrotmike/CS909/blob/master/pytorch_nn_barebones.ipynb
- Solve Digit Classification (demonstrates tensorboard use)
- https://github.com/foxtrotmike/CS909/blob/master/pytorch_mlp_mnist.ipynb

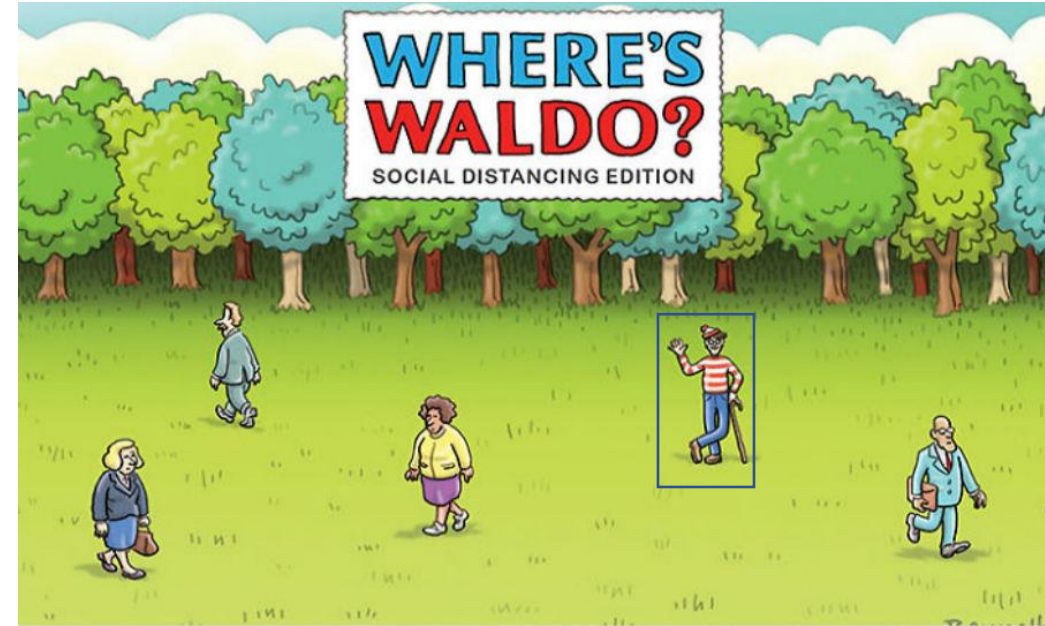
Where's Waldo?

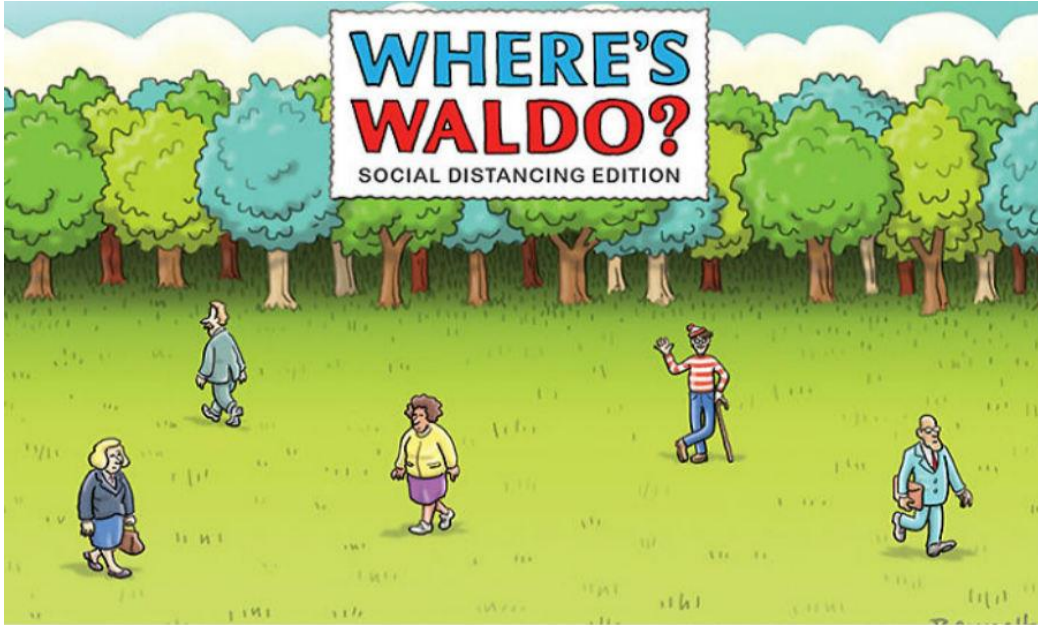






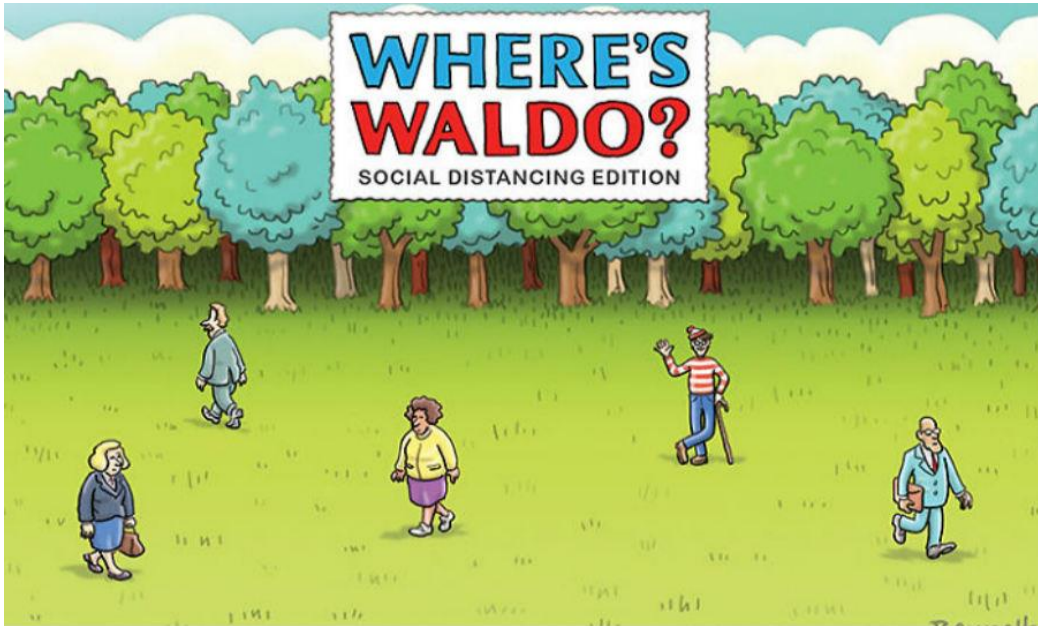
odel



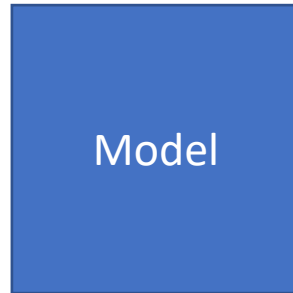


we use a perceptron here then what would be the number of weights needed if the image size of the input and output is $N \times N$?

Even single layer
 $O(N \times N \times N \times N)$

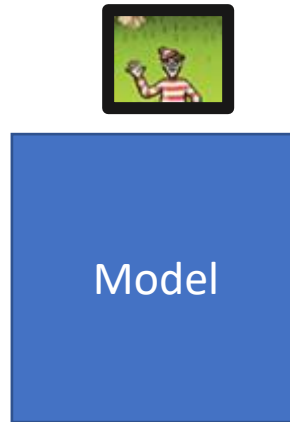
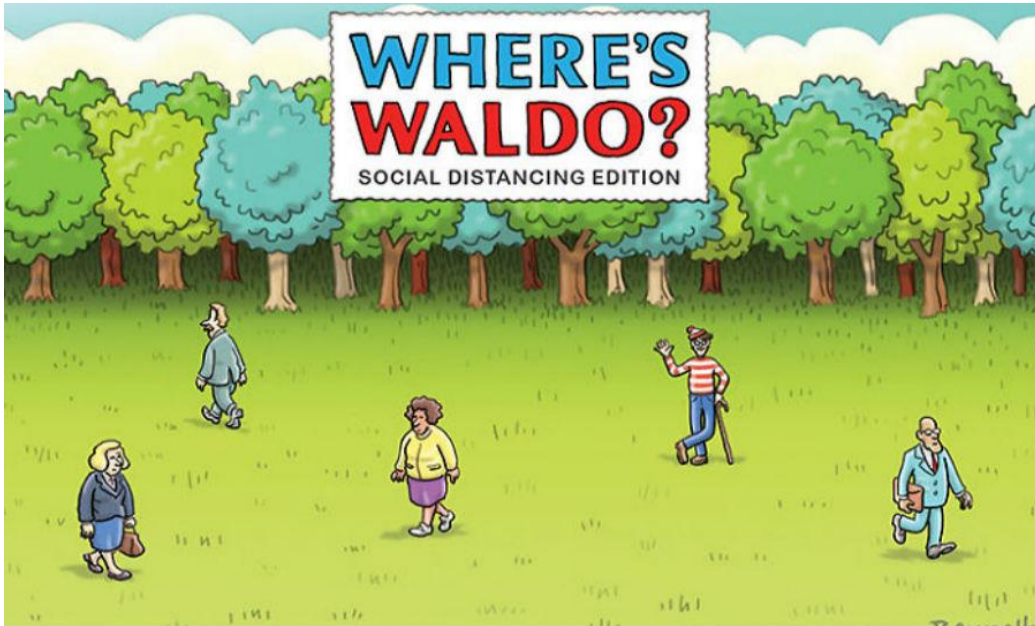


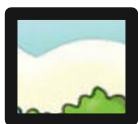
$N \times N$



$N \times N$

- Assume we have a cutout of what Waldo looks like
- And if we “scan” (formally called correlate or convolve) the cutout against the input image – we should see a peak at the location where Waldo occurs in the input image
- Correlation is dot-product (a measure of similarity)





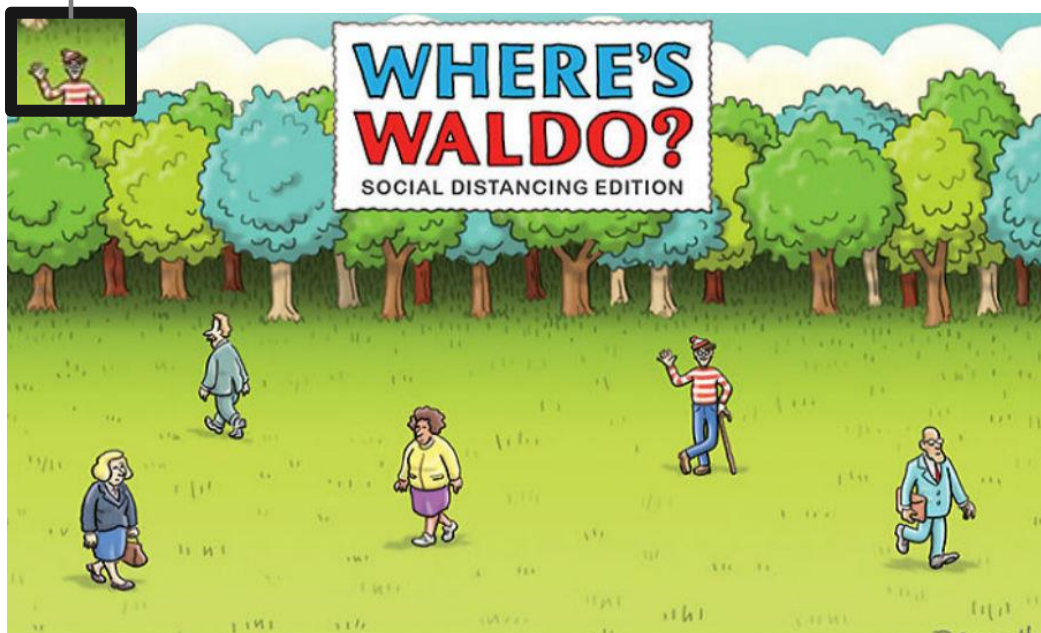
0.28	0.32	0.24	0.91	0.78
0.86	0.66	0.89	0.51	0.4
0.53	0.3	0.41	0.17	0.25
0.4	0.47	0.66	0.11	0.94
0.19	0.73	0.097	0.7	0.37

$$\begin{bmatrix} 0.28 \\ 0.32 \\ \vdots \\ 0.37 \end{bmatrix}$$

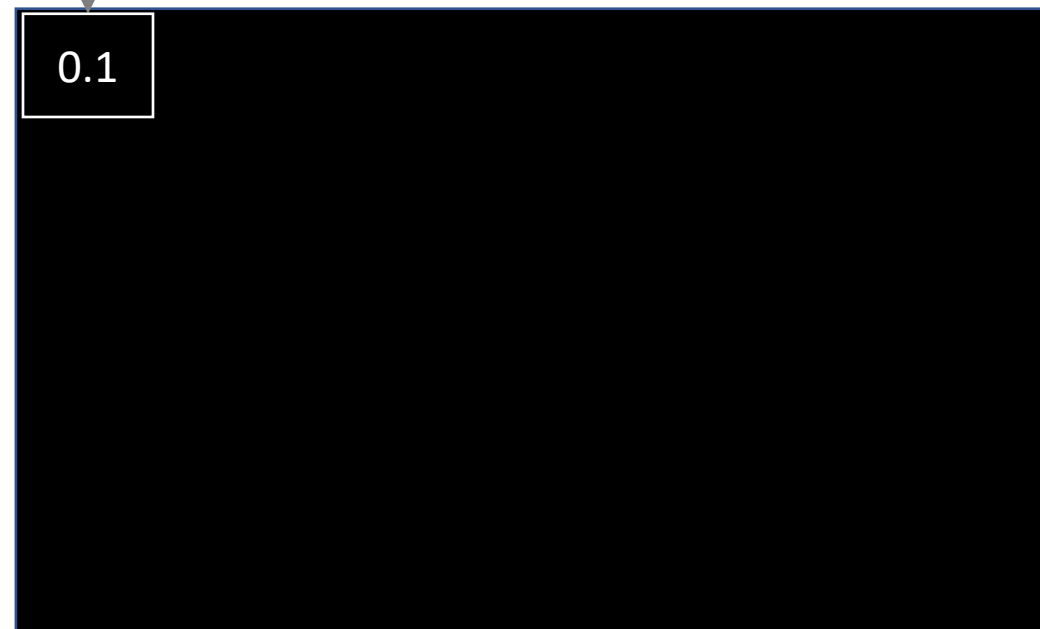
•

$$\begin{bmatrix} 0.79 \\ 0.32 \\ \vdots \\ 0.37 \end{bmatrix}$$

0.79	0.32	0.9	0.47	0.88
0.86	0.57	0.16	0.39	0.013
0.93	0.53	0.74	0.67	0.82
0.41	0.7	0.011	0.26	0.35
0.74	0.68	0.93	0.19	0.37



Model





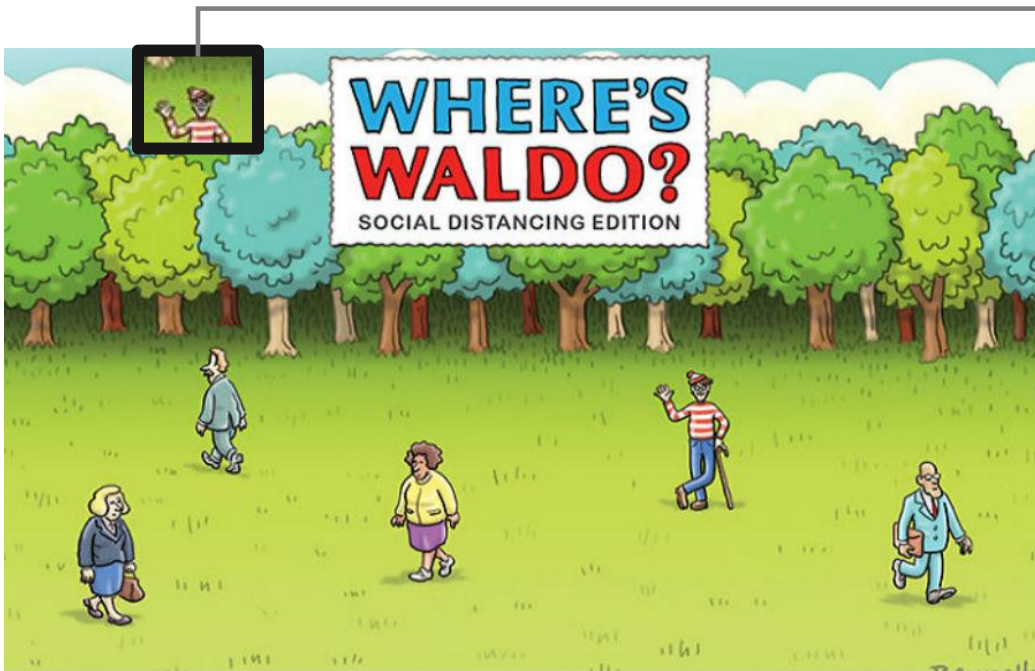
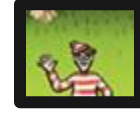
0.45	0.68	0.87	0.39	0.19
0.2	0.12	0.9	0.63	0.38
0.99	0.86	0.4	0.82	0.28
0.36	0.98	0.8	0.065	0.58
0.28	0.21	0.067	0.25	0.36

$$\begin{bmatrix} 0.45 \\ 0.68 \\ \vdots \\ 0.36 \end{bmatrix}$$

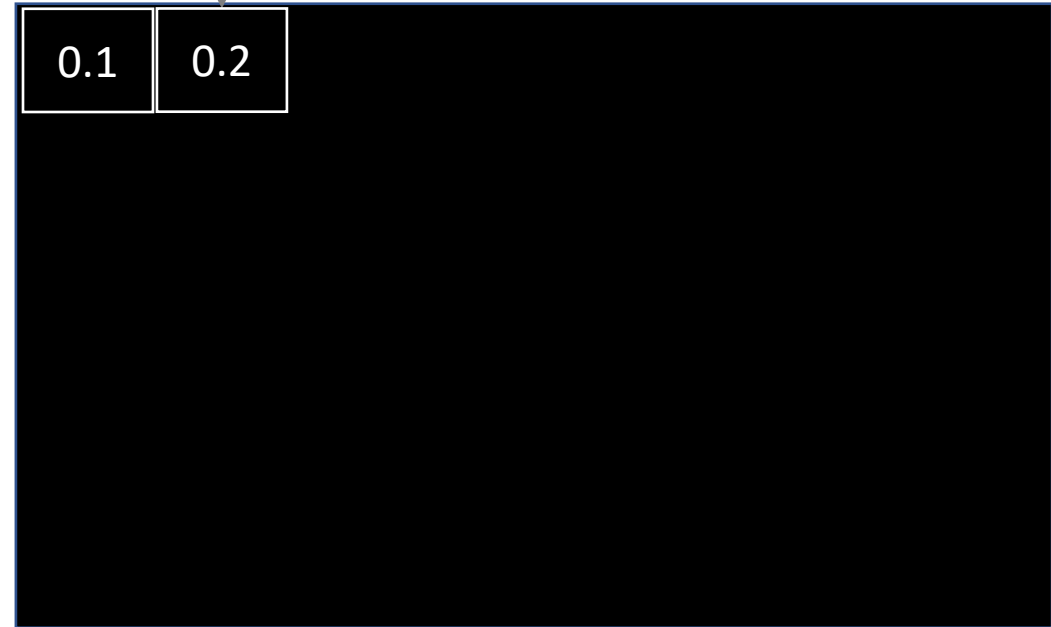
•

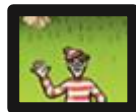
$$\begin{bmatrix} 0.79 \\ 0.32 \\ \vdots \\ 0.37 \end{bmatrix}$$

0.79	0.32	0.9	0.47	0.88
0.86	0.57	0.16	0.39	0.013
0.93	0.53	0.74	0.67	0.82
0.41	0.7	0.011	0.26	0.35
0.74	0.68	0.93	0.19	0.37



Model





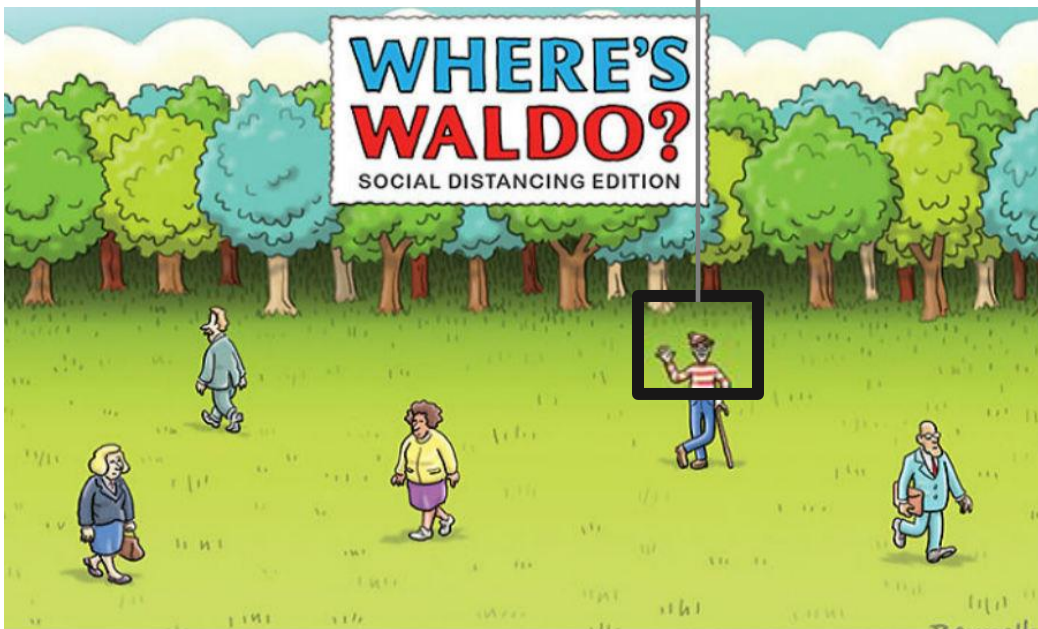
0.79	0.32	0.9	0.47	0.88
0.86	0.57	0.16	0.39	0.013
0.93	0.53	0.74	0.67	0.82
0.41	0.7	0.011	0.26	0.35
0.74	0.68	0.93	0.19	0.37

$$\begin{bmatrix} 0.79 \\ 0.32 \\ \vdots \\ 0.37 \end{bmatrix}$$

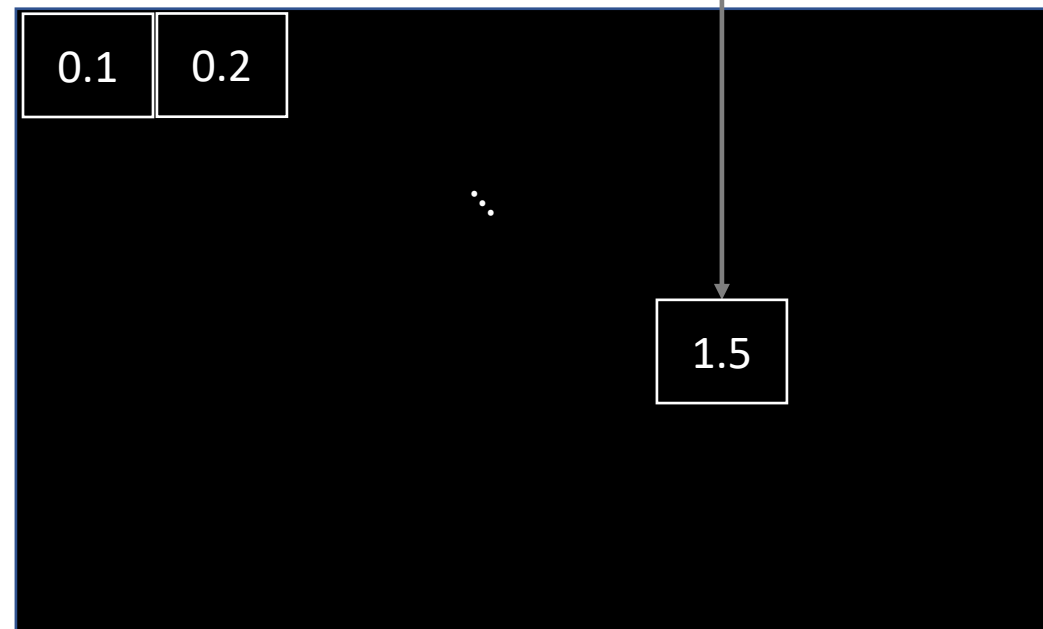
•

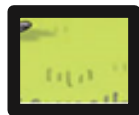
$$\begin{bmatrix} 0.79 \\ 0.32 \\ \vdots \\ 0.37 \end{bmatrix}$$

0.79	0.32	0.9	0.47	0.88
0.86	0.57	0.16	0.39	0.013
0.93	0.53	0.74	0.67	0.82
0.41	0.7	0.011	0.26	0.35
0.74	0.68	0.93	0.19	0.37



Model





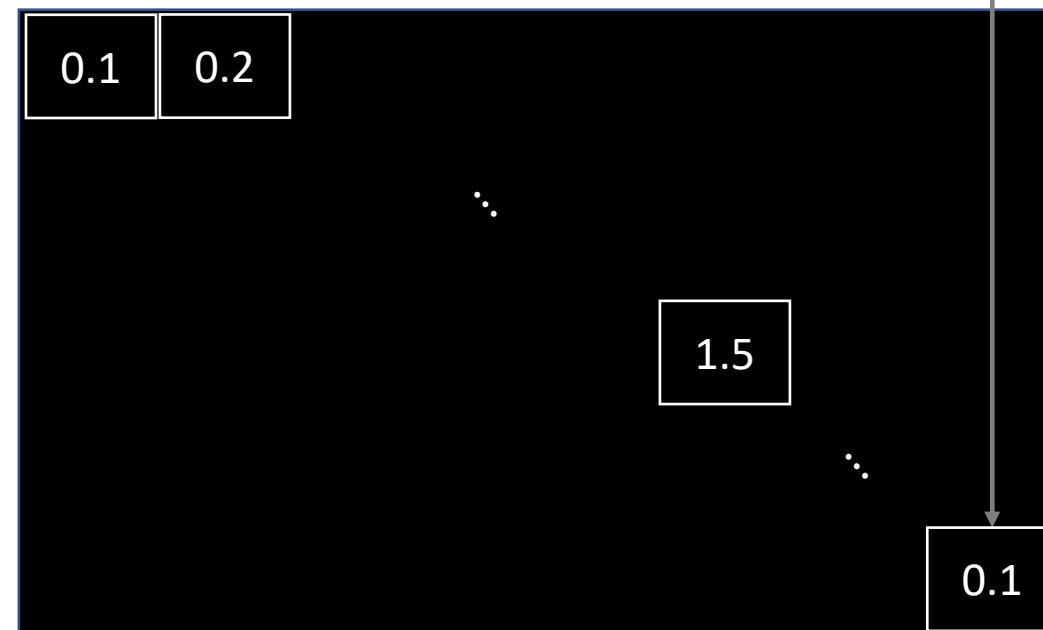
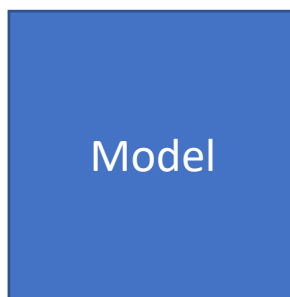
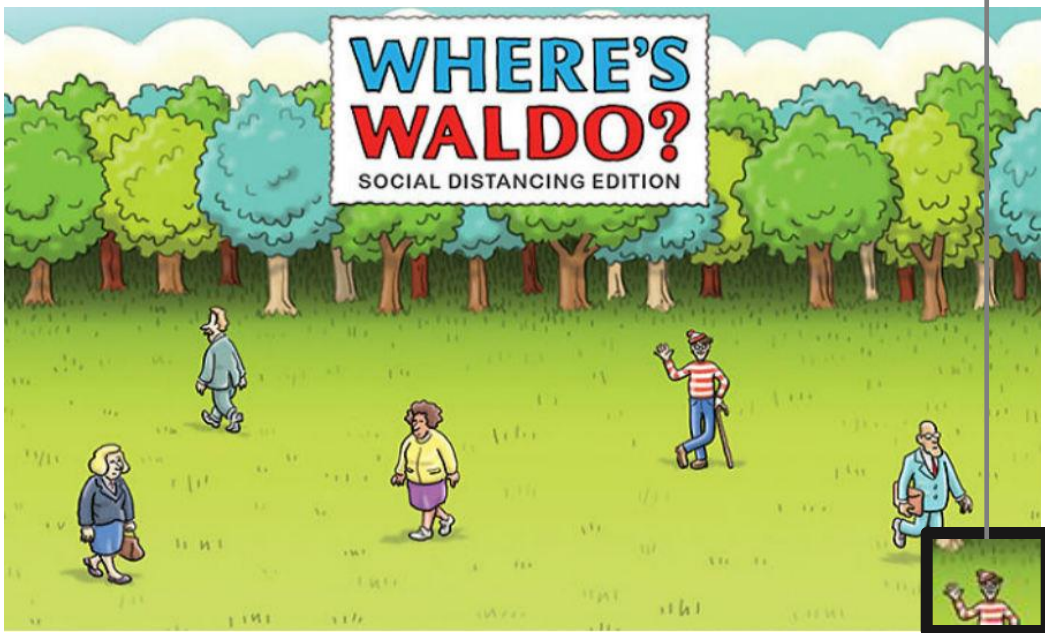
0.1	0.7	0.27	0.98	0.96
0.58	0.0054	0.34	0.26	0.025
0.31	0.98	0.54	0.89	0.54
0.67	0.05	0.24	0.15	0.83
0.83	0.48	0.33	0.42	0.67

$$\begin{bmatrix} 0.10 \\ 0.70 \\ \vdots \\ 0.67 \end{bmatrix}$$

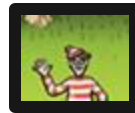
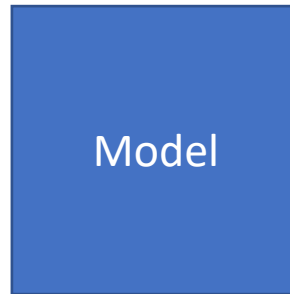
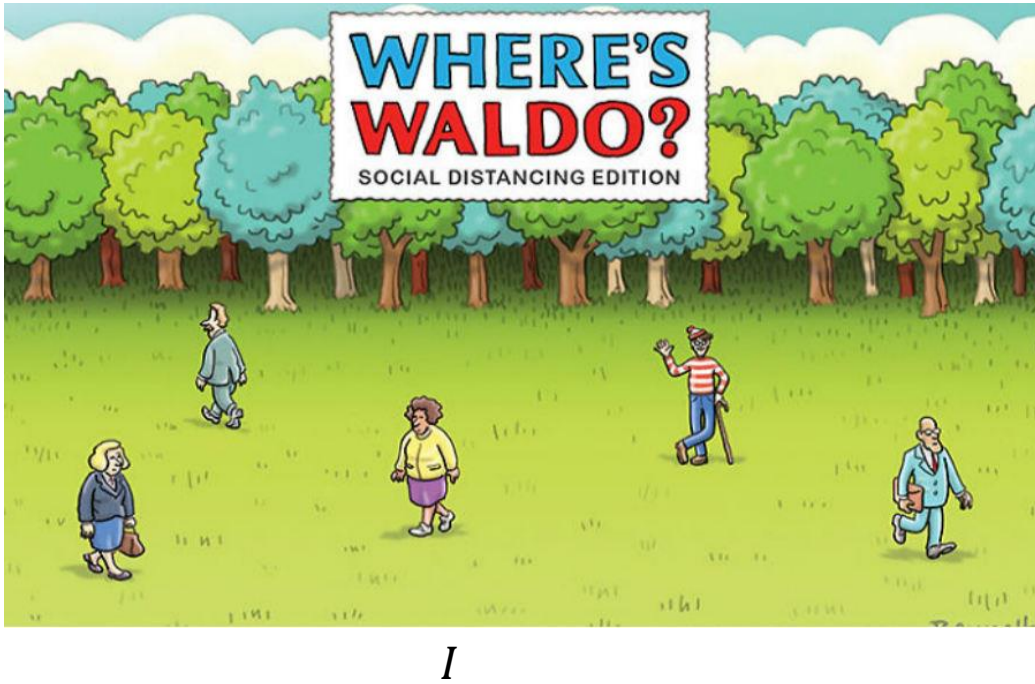
•

$$\begin{bmatrix} 0.79 \\ 0.32 \\ \vdots \\ 0.37 \end{bmatrix}$$

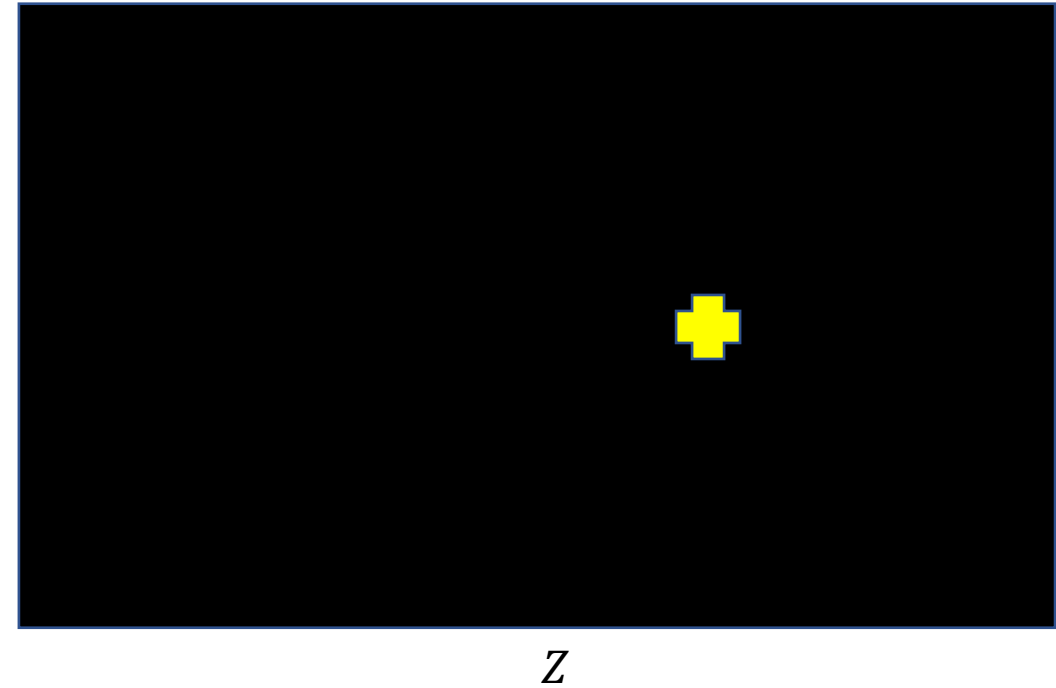
0.79	0.32	0.9	0.47	0.88
0.86	0.57	0.16	0.39	0.013
0.93	0.53	0.74	0.67	0.82
0.41	0.7	0.011	0.26	0.35
0.74	0.68	0.93	0.19	0.37



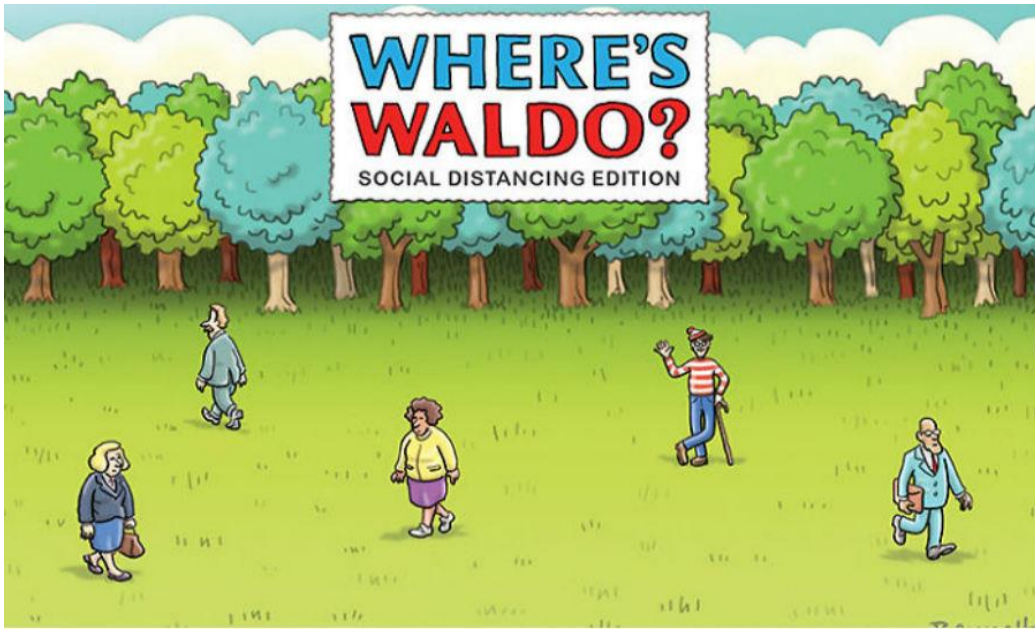
- Output of convolution (strictly speaking – correlation) with the template “filter” will have a peak at the location of the most similar object
- Mathematically, $Z = I * W$



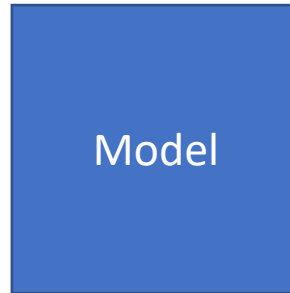
W



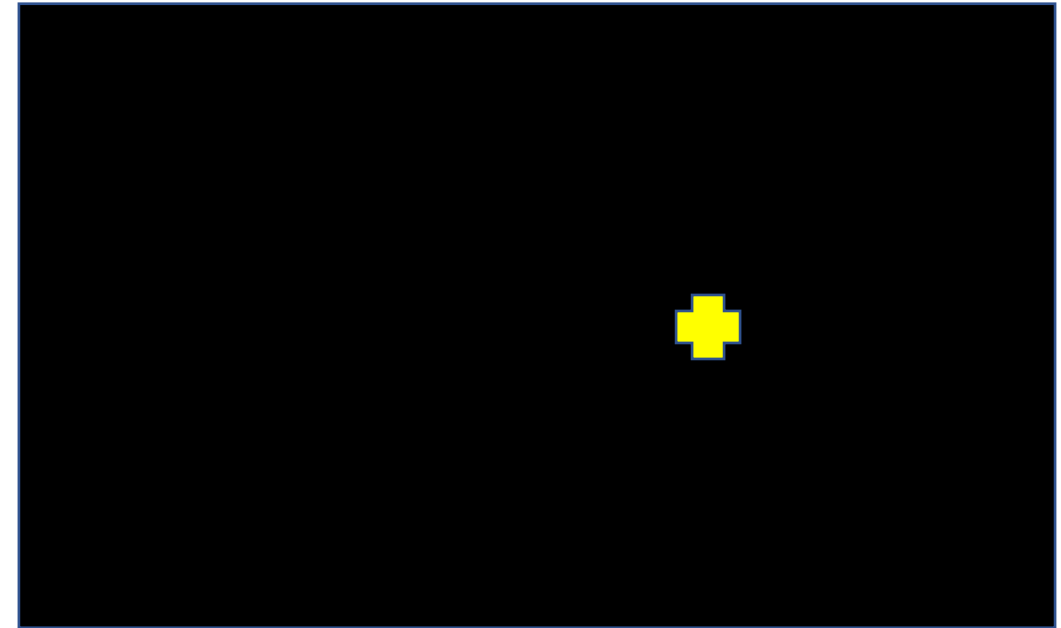
- What if I don't have a cut out of Waldo but I do have a target image specifying where the peak should be?
- Can we find him still?



I

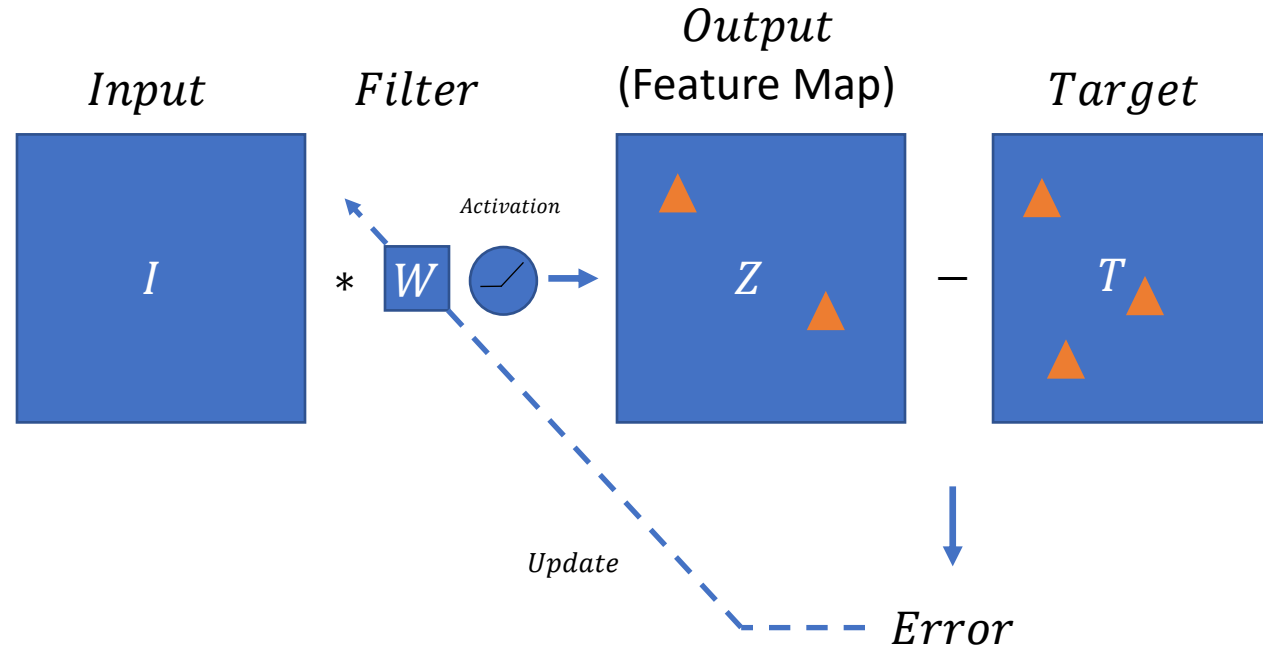


$W = ?$



Z

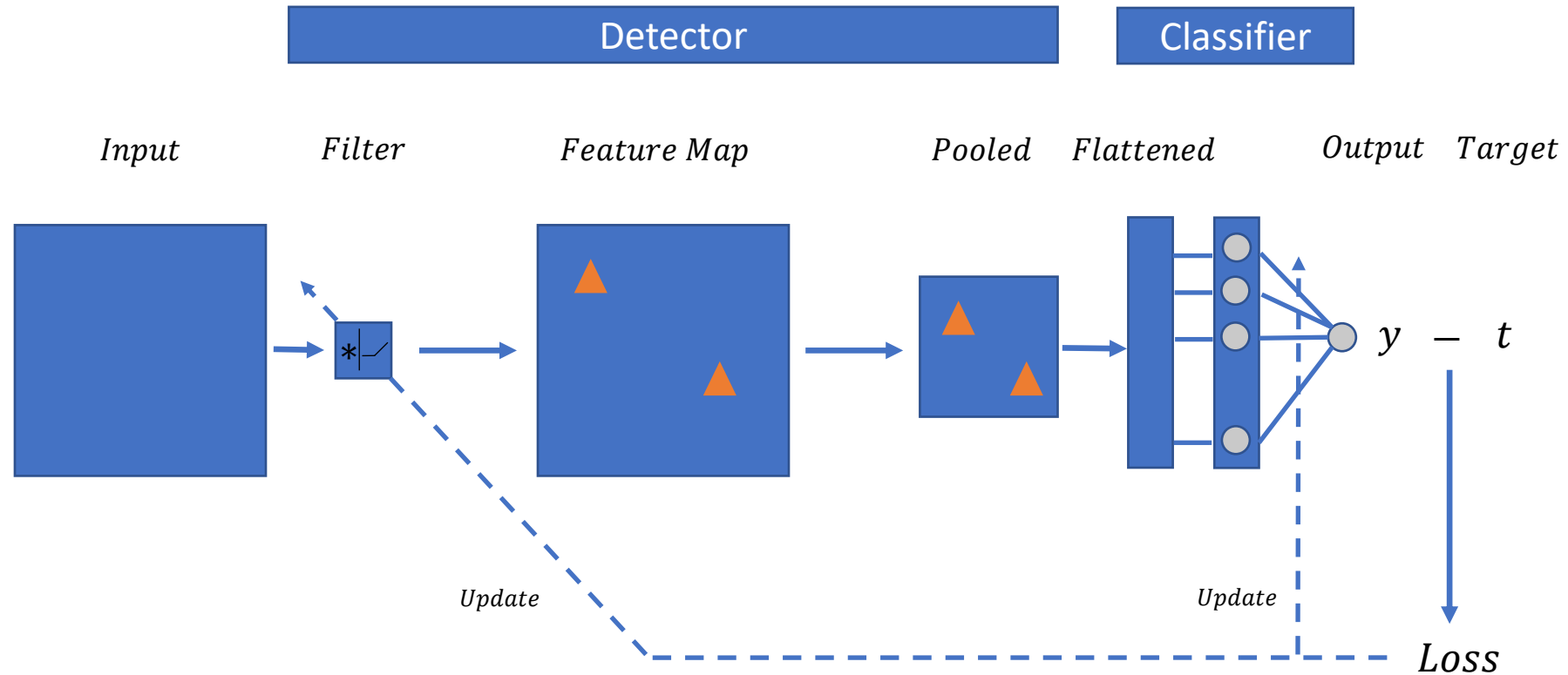
(Very) basic “convolutional neural network”



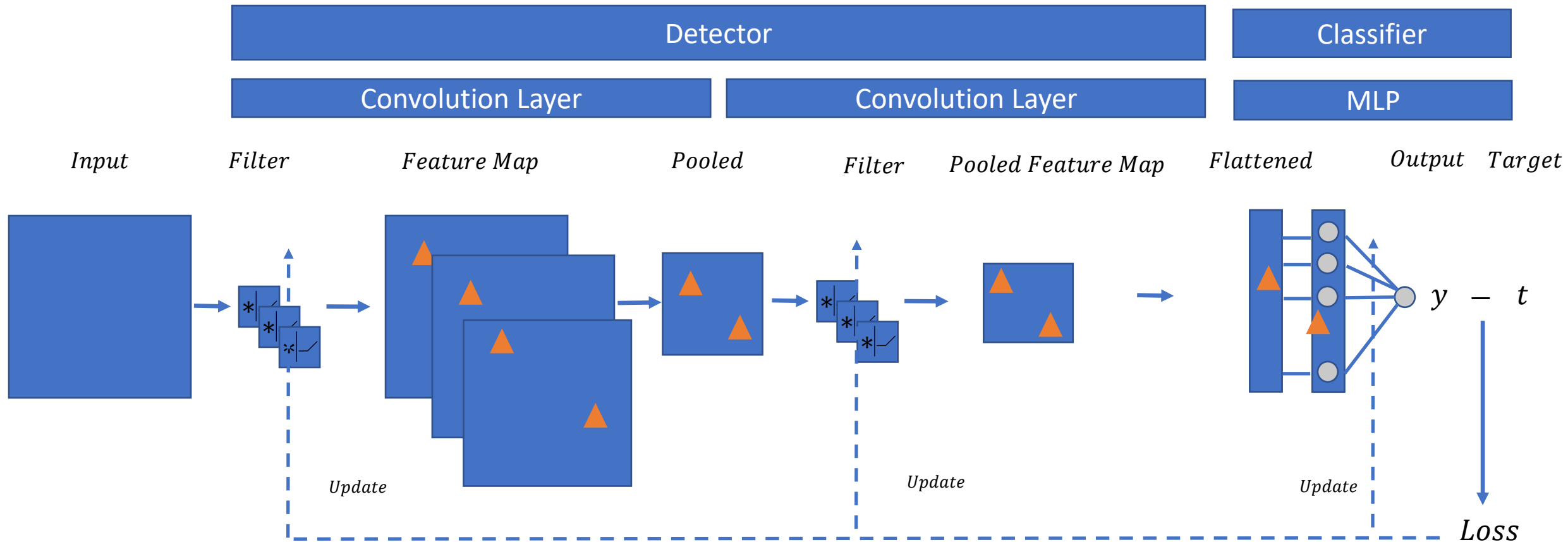
- Technically, this is a single layer in a Fully Convolutional Neural Network
 - **Intuition:** Each “filter” learns to detect a specific “local” pattern
- The **representation** of a convolutional layer is: $Z = a(I * W)$ different from MLPs
 - More efficient as W is much smaller
- The complete learning problem with **Evaluation** and **Optimization** is: $\min_W \|a(I * W) - T\|$

MLP Layer: $Z = a(WX)$
Convolutional Layer: $Z = a(I * W)$

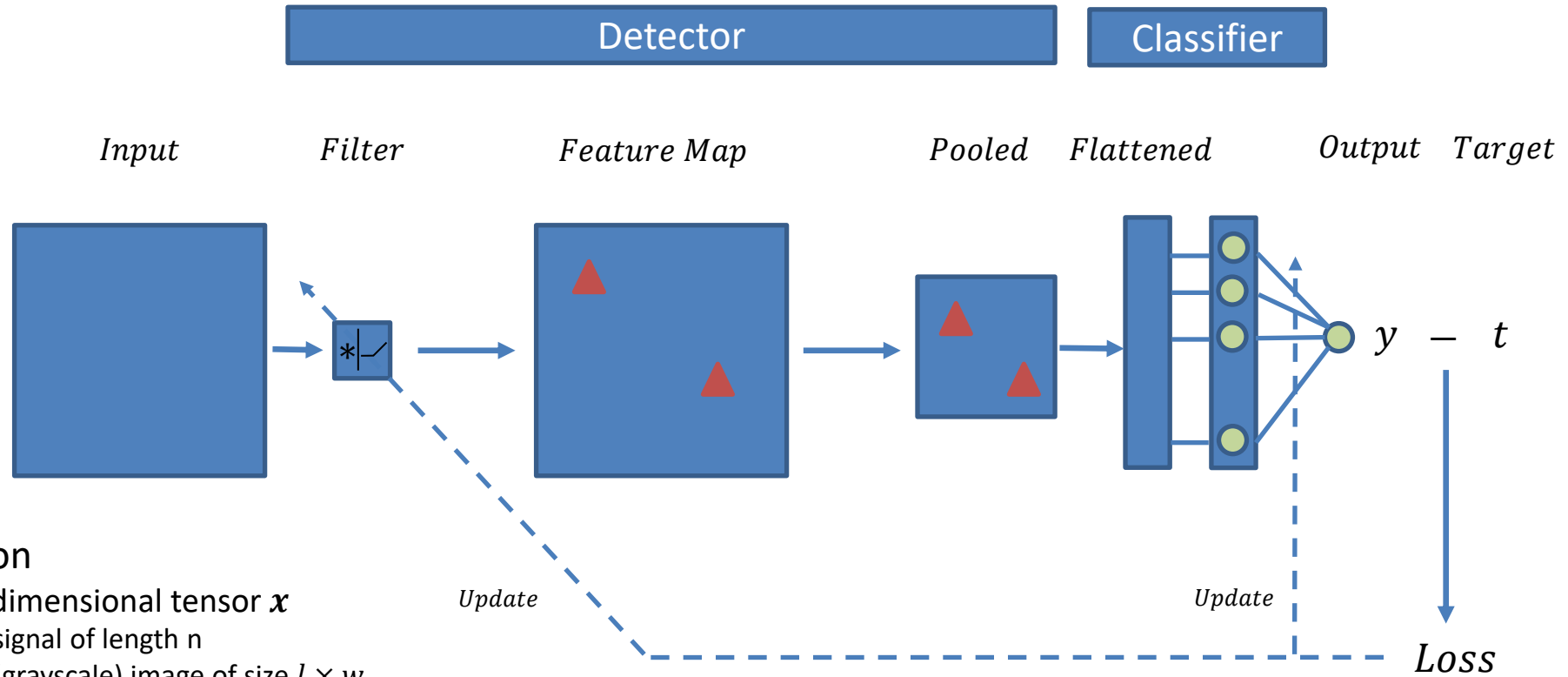
If we want to classify images...



CNNs



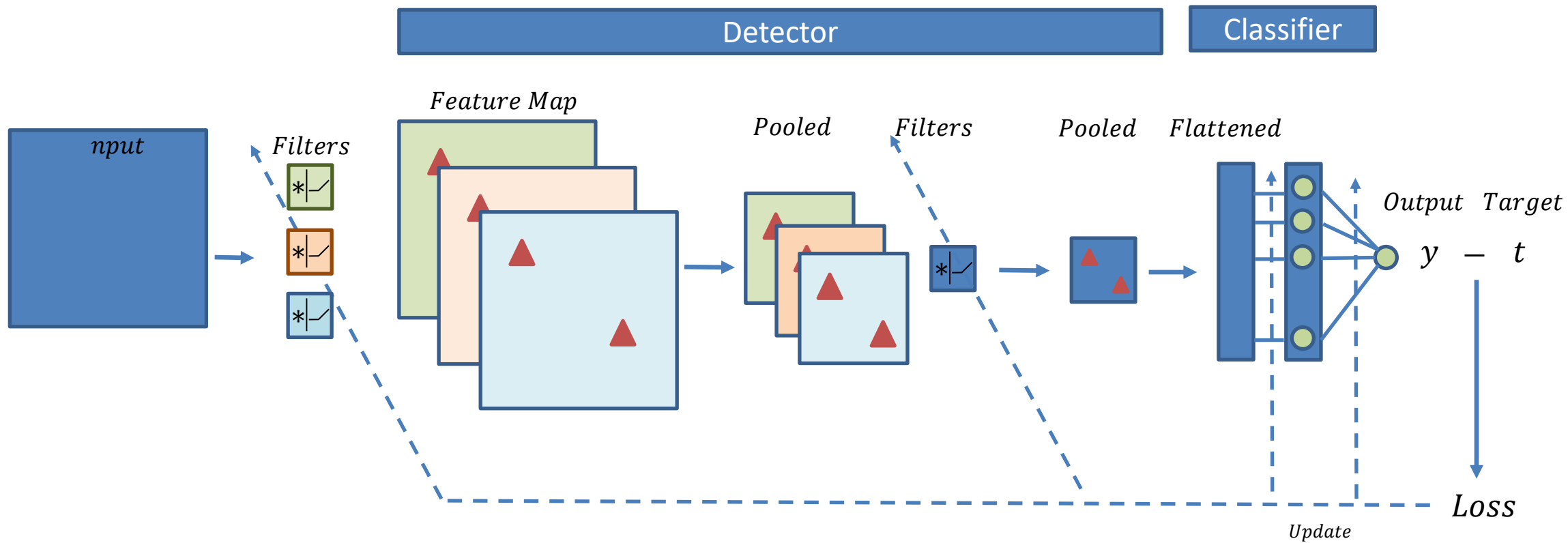
EO for a convolution neural network



- **Representation**

- Input: a k -dimensional tensor x
 - $k = 1$: signal of length n
 - $k = 2$: (grayscale) image of size $l \times w$
 - RGB channel image: $l \times w \times 3$
 - $k = 3$: $l \times w \times t$ video of frame size $l \times w$ with duration t
- Output: A decision score $y = f(x; \theta)$ (can be multi-dimensional as well)
- Structure

- **Layers of Learnable filters each of which is correlated (or convolved) with the input tensor in parallel followed by convolution with other filters**
 - A single convolution is indicated by $z = \alpha(x \star \theta)$ where θ is the representation of a single filter and $\alpha(\cdot)$ is an activation function. Filters are much smaller than x .
 - Implemented as layers: Conv1d, Conv2d, Conv3d (in PyTorch)
- The correlation output is then pooled (optional)
- Nonlinear activation functions are applied
- Aggregated to produce the final output (depending upon application)



Lab Exercise

- The concept of filters, convolution and learning filters in PyTorch leading to the simplest possible convolutional neural network
- https://github.com/foxtrotmike/CS909/blob/master/learn_filters.ipynb
- Using Convolutional neural networks in Pytorch for digit classification
- https://github.com/foxtrotmike/CS909/blob/master/cnn_mnist_pytorch.ipynb